Technical Report — E-Commerce Database Project

MTM4692 — SQL Project

Team Members: - Erhan Türker - Andaç Demirdelen - Hüseyin Ekşici - Ahmet Barkın Mecer

Repository: github.com/erhantrk/MTM4692_Project **Presentation"" [https://youtu.be/1zZWDn2IHEw]

---

1. Problem Statement

Small and medium e-commerce businesses collect large amounts of transactional data — customer registrations, product listings, orders, payments, and reviews — but often lack a structured system to store, query, and analyze this information. Without a well-designed relational database, critical business questions go unanswered: Which products drive the most revenue? Which customers are the most valuable? Are certain product categories underperforming? What does monthly revenue growth look like?

This project addresses that gap by designing a normalized relational database for an online retail store. The database captures the full lifecycle of an e-commerce transaction — from the customer browsing a product catalog, to placing an order with multiple items, to making a payment, and finally leaving a review. The goal is to create a schema that supports efficient data storage and powerful analytical queries.

2. Why It Matters

E-commerce is one of the fastest-growing sectors in the global economy. Even a small online store generates hundreds or thousands of records per month across customers, orders, and products. The ability to query this data effectively translates directly into better business decisions:

- **Revenue optimization**: Identifying top-selling products and high-value customers allows targeted marketing and inventory planning.
- **Customer retention**: Tracking purchase history and review patterns helps detect churn risk and reward loyalty.
- **Operational efficiency**: Monitoring stock quantities, order volumes, and payment statuses prevents overselling and delays.
- **Product quality**: Aggregating review scores highlights products that need improvement or removal.

Feasibility: The scope of this project — 7 tables, 5–7 key queries, and 3 views — is well-suited for a semester-long course project. The domain is familiar and

well-defined, the relationships between entities are natural and unambiguous, and the data lends itself to a wide variety of SQL techniques (joins, aggregation, CTEs, subqueries, and views). This makes the project both manageable and rich enough to demonstrate meaningful database skills.

3. Preliminary Schema

3.1 `customers`

Stores registered customer accounts.

Column	Data Type	Constraints
customer_id	INTEGER	PRIMARY KEY, AUTO_INCREMENT
first_name	TEXT	NOT NULL
last_name	TEXT	NOT NULL
email	TEXT	NOT NULL, UNIQUE
phone	TEXT	
address	TEXT	
city	TEXT	
state	TEXT	
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

3.2 `categories`

Groups products into browsable categories.

Column	Data Type	Constraints
category_id	INTEGER	PRIMARY KEY, AUTO_INCREMENT
name	TEXT	NOT NULL, UNIQUE
description	TEXT	

3.3 `products`

The product catalog, linked to categories.

Column	Data Type	Constraints
product_id	INTEGER	PRIMARY KEY, AUTO_INCREMENT
category_id	INTEGER	FOREIGN KEY → categories(category_id)
name	TEXT	NOT NULL
description	TEXT	
price	INTEGER	NOT NULL, CHECK (price > 0)
stock_qty	INTEGER	NOT NULL, DEFAULT 0

Column	Data Type	Constraints
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

3.4 `orders`

Records each purchase order placed by a customer.

Column	Data Type	Constraints
order_id	INTEGER	PRIMARY KEY, AUTO_INCREMENT
customer_id	INTEGER	FOREIGN KEY → customers(customer_id)
order_date	DATETIME	DEFAULT CURRENT_TIMESTAMP
status	TEXT	NOT NULL, DEFAULT 'pending'
total_amount	REAL	NOT NULL

3.5 `order_items`

Line items within an order — the junction table between orders and products.

Column	Data Type	Constraints
item_id	INTEGER	PRIMARY KEY, AUTO_INCREMENT
order_id	INTEGER	FOREIGN KEY → orders(order_id)
product_id	INTEGER	FOREIGN KEY → products(product_id)
quantity	INTEGER	NOT NULL, CHECK (quantity > 0)
unit_price	REAL	NOT NULL

3.6 `payments`

Payment records for each order.

Column	Data Type	Constraints
payment_id	INTEGER	PRIMARY KEY, AUTO_INCREMENT
order_id	INTEGER	FOREIGN KEY → orders(order_id)
payment_date	DATETIME	DEFAULT CURRENT_TIMESTAMP
amount	REAL	NOT NULL
payment_method	TEXT	NOT NULL

3.7 `reviews`

Customer reviews for products they have purchased.

Column	Data Type	Constraints
review_id	INTEGER	PRIMARY KEY, AUTO_INCREMENT
customer_id	INTEGER	FOREIGN KEY → customers(customer_id)
product_id	INTEGER	FOREIGN KEY → products(product_id)
rating	INTEGER	NOT NULL, CHECK (rating BETWEEN 1 AND 5)
comment	TEXT	
review_date	DATETIME	DEFAULT CURRENT_TIMESTAMP

4. Proposed SQL Features

The final project will demonstrate the following SQL techniques:

Feature	Planned Usage
Joins	Combine customers, orders, order_items, and products to build full order reports
Aggregation	SUM, COUNT, AVG for revenue totals, order counts, average ratings
Views	3 views — monthly revenue summary, top customers, product performance
CTEs	Rank customers by lifetime value; identify products above average revenue
Subqueries	Find customers who have never left a review; products with no orders

Planned Views (3)

1. `vw_monthly_revenue` — Aggregates total revenue by month.
2. `vw_top_customers` — Ranks customers by total spending using a CTE.
3. `vw_product_performance` — Shows each product's total sales, average rating, and review count.

Planned Key Queries (5–7)

1. Monthly revenue report (aggregation + group by)
2. Top 10 customers by lifetime spending (CTE + join + aggregation)
3. Best and worst reviewed products (join + aggregation + order by)
4. Customers who have never left a review (subquery with NOT IN / NOT EXISTS)
5. Revenue breakdown by product category (multi-table join + aggregation)
6. Products with above-average revenue (subquery or CTE)
7. Order details report joining customers, orders, items, products, and payments

5. Sample Queries

Query 1 — Monthly Revenue Report (Aggregation + Join)

```
SELECT
    DATE_FORMAT(o.order_date, '%Y-%m')  AS month,
    COUNT(DISTINCT o.order_id)          AS total_orders,
    SUM(o.total_amount)                 AS total_revenue
FROM orders o
WHERE o.status != 'cancelled'
GROUP BY DATE_FORMAT(o.order_date, '%Y-%m')
ORDER BY month DESC;
```

Query 2 — Top 5 Customers by Lifetime Value (CTE + Join + Aggregation)

```
WITH customer_spending AS (
    SELECT
        o.customer_id,
        SUM(o.total_amount)  AS lifetime_value,
        COUNT(o.order_id)    AS order_count
    FROM orders o
    WHERE o.status != 'cancelled'
    GROUP BY o.customer_id
)
SELECT
    c.first_name,
    c.last_name,
    c.email,
    cs.lifetime_value,
    cs.order_count
FROM customer_spending cs
JOIN customers c ON cs.customer_id = c.customer_id
ORDER BY cs.lifetime_value DESC
LIMIT 5;
```

Query 3 — Products with No Reviews (Subquery)

```
SELECT
    p.product_id,
    p.name,
    p.price,
    cat.name AS category
FROM products p
JOIN categories cat ON p.category_id = cat.category_id
WHERE p.product_id NOT IN (
    SELECT DISTINCT product_id
```

```
    FROM reviews
);
```